

# Processes

Finding, Monitoring and Managing | Scott Pattison

## 1. Finding a Running Process

### ps aux | grep jellyfin

Breaking it down:

```
ps = command process snapshot - show a snapshot of all current running processes.
aux = options
a: Shows the processes for all users, not just you.
u: Display in user-oriented format. Adds columns / headers
x: Includes processes not attached to this terminal, background daemons,
services, anything running without a visible terminal window.
| = a pipe = passes the output from ps aux over to the next command.
grep jellyfin = Global Regular Expression Print. Searches through text
(the piped output), printing only lines containing 'jellyfin'
```

■■ This command will **NOT** print the column headers — the output from ps aux is passed to grep which filters out the header row as it does not contain 'jellyfin'. If grep finds no match, nothing is printed at all.

### To see headers alongside the results:

```
ps aux | grep -E "USER|jellyfin"
```

-E unlocks extended pattern matching. Without it, grep searches for the literal string 'USER|jellyfin'. With -E, the | becomes an OR operator:

| Symbol | Meaning in -E mode                      |
|--------|-----------------------------------------|
|        | OR - match this pattern OR that pattern |
| +      | One or more of the previous character   |
| ?      | Zero or one of the previous character   |
| ()     | Grouping                                |

So `grep -E "USER|jellyfin"` searches the ps aux output and prints every line containing USER, or jellyfin, or both.

## 2. ps -ef — An Alternative to ps aux

**ps -ef** shows the same information as **ps aux**, but uses a different syntax style and returns a slightly different output format.

### Why are there two ways to do the same thing?

- **BSD style** – ps aux (no dashes) - came from BSD Unix
- **UNIX/POSIX style** – ps -ef (with dashes) – came from System V Unix

UNIX was created in 1969 at Bell Labs (AT&T) — the original operating system everything else grew from. In the 1980s AT&T standardised UNIX under **POSIX** (Portable Operating System Interface) — a set of rules so software written for one system would work on another.

**BSD** (Berkeley Software Distribution) — developed at the University of California, Berkeley in the late 1970s–80s. Started as additions to UNIX, became its own OS. Introduced TCP/IP networking, vi editor, and virtual memory among others.

|                    | UNIX / POSIX                            | BSD                                         |
|--------------------|-----------------------------------------|---------------------------------------------|
| Command Style      | Options use dashes: <code>ps -ef</code> | Options without dashes: <code>ps aux</code> |
| Origin             | Bell Labs / AT&T;                       | University of California, Berkeley          |
| Modern descendants | Linux, AIX, Solaris                     | FreeBSD, OpenBSD, macOS                     |

### What about Windows OS

Windows has a completely separate origin from Unix/BSD. Created by Microsoft in 1985 as a graphical shell on top of MS-DOS. MS-DOS was based on QDOS (Quick and Dirty Operating System).

### Why does Linux use both?

Linux was written from scratch by Linus Torvalds but drew heavily from both traditions. Rather than choosing one style, Linux supports both — which is why **ps aux (BSD)** and **ps -ef (UNIX/POSIX)** both work on modern systems.

### PPID — the key difference

**ps -ef** includes a PPID column (Parent Process ID) which **ps aux** does not. PPID shows which process spawned this one — useful for understanding parent/child relationships.

#### ps -ef columns (PPID highlighted):

|     |     |      |     |       |     |      |     |
|-----|-----|------|-----|-------|-----|------|-----|
| \$1 | \$2 | \$3  | \$4 | \$5   | \$6 | \$7  | \$8 |
| UID | PID | PPID | C   | STIME | TTY | TIME | CMD |

|                            | ps aux                  | ps -ef                  |
|----------------------------|-------------------------|-------------------------|
| Style                      | BSD                     | UNIX/POSIX              |
| Dashes                     | No dashes               | Dashes required         |
| Shows all users            | Yes                     | Yes                     |
| Shows background processes | Yes                     | Yes                     |
| Shows process state (STAT) | Yes                     | No                      |
| Shows PPID                 | No                      | Yes                     |
| Shows %CPU and %MEM        | Yes                     | No                      |
| Output format              | Different column layout | Different column layout |

In practice, use **ps aux** for process states or CPU/memory usage, and **ps -ef** to identify parent/child relationships via PPID.

### 3. The ps aux Headers

Explanation of headers from a real world example of `ps aux | grep -E "USER|jellyfin":`

| Header  | Meaning                                                                       |
|---------|-------------------------------------------------------------------------------|
| USER    | Who owns the process                                                          |
| PID     | Process ID - unique identifier                                                |
| %CPU    | Current CPU usage                                                             |
| %MEM    | Current Memory usage (See %MEM/RSS/VSZ section below)                         |
| VSZ     | Virtual Memory Size (See below)                                               |
| RSS     | Actual Physical Memory used (See below)                                       |
| TTY     | Which Terminal it is attached to (See TTY section below)                      |
| STAT    | Process state: R=Running, S=Sleeping, Z=Zombie etc (See Process States below) |
| START   | When it started                                                               |
| TIME    | Total CPU time used                                                           |
| COMMAND | The actual command that launched it                                           |

#### STAT = Process States

Every process on the system is always in one of the following states:

| State                 | Code | Meaning                                                                                                       |
|-----------------------|------|---------------------------------------------------------------------------------------------------------------|
| Running               | R    | Currently executing on the CPU, or in the queue ready to run                                                  |
| Sleeping              | S    | Waiting for something — user input, a file, a network response. Most processes spend most of their time here. |
| Stopped               | T    | Paused — usually by a signal like Ctrl+Z. Can be resumed                                                      |
| Zombie                | Z    | Finished executing but not yet cleaned up by its parent process                                               |
| Uninterruptible Sleep | D    | Waiting on hardware I/O (disk, network) — cannot be interrupted or killed until the operation completes       |

■■ **Please Note:** A Zombie process cannot be killed directly - it is already dead. The Zombie entry remains in the process table waiting for its parent to collect its exit status. To clean it up you will need to kill the **parent** process.

#### First lets find all of the Zombies:

```
ps aux | awk '$8 == "Z"' # Matches Z in column 8 - the STAT column
```

**awk** is a standalone text processing command that works on **columns** rather than lines. Where **grep** searches anywhere in a line, **awk** targets specific fields. `'$8 == "Z"'` is the instruction given to **awk**:

- `$8` — the 8th column (awk numbers from \$1)
- `==` — equals comparison
- `"Z"` — the value to match
- Together: "only print this line if column 8 equals Z"

### Other awk examples:

| Example                            | What it does                                      |
|------------------------------------|---------------------------------------------------|
| awk '{print \$1}' file             | Print only the first column of every line         |
| awk '\$3 > 50' file                | Print lines where column 3 is greater than 50     |
| awk -F: '{print \$1}' /etc/passwd  | Use : as delimiter, print first field (usernames) |
| awk '{sum += \$3} END {print sum}' | Add up all values in column 3                     |
| ps aux   awk '\$3 > 10'            | Show processes using more than 10% CPU            |
| df -h   awk '\$5 > "80%"'          | Show filesystems over 80% full                    |

### Killing the Zombies!

Once you have found the zombies use the following command to kill the parent and in doing so clean up the zombie:

```
kill -9 # Send SIGKILL to the parent process, forcibly terminating it.
# When the parent dies, the kernel automatically cleans up
# any zombie children it left behind.
```

### TTY Values from the Header Explanations

| TTY Value           | Meaning                                                                                                                                                    |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ?                   | No terminal - background daemon/service (Docker, SSH, Jellyfin)                                                                                            |
| tty 1 to tty 6      | Virtual consoles - what you would see if you plugged hardware into the server. Note: TTYs are created on a headless server anyway — software placeholders. |
| pts/1, pts/2<br>etc | Pseudo Terminal Slave — SSH sessions and tmux panes. Each connection gets its own pts number.                                                              |
| tty 7               | Usually where the GUI runs, if one is installed.                                                                                                           |

### %MEM, RSS and VSZ (from the header explanations)

When looking at process memory usage there are three columns to understand. **RSS (Resident Set Size)** is the actual physical RAM the process is using right now, measured in kilobytes — this is the real number that matters. **%MEM** is simply RSS expressed as a percentage of your total RAM — not a separate measurement, just a calculation. **VSZ (Virtual Memory Size)** is the total memory the process has reserved, including RAM, swap space, shared libraries, and memory mapped files — always larger than RSS and often alarmingly high, but don't panic. A process reserves more than it actually uses. When checking memory consumption, **RSS and %MEM are what matter** — VSZ is largely misleading.

## 4. The kill Command and Signals

The kill command is used to send signals to processes. Despite its name it does not just terminate processes — it can pause them, resume them, or ask them to reload their configuration.

### Why would you use it?

There will be times when a process stops responding, is consuming too many resources, or is behaving unexpectedly — you need a way to stop it or control it. There will also be times when you change a service's config file and need to tell it to reload without restarting. Understanding signals explains why sometimes you need a forced termination and sometimes a gentler approach is the right choice.

| Basic Syntax         | What it does                               |
|----------------------|--------------------------------------------|
| <code>kill</code>    | Send default signal (SIGTERM) to a process |
| <code>kill -9</code> | Send SIGKILL - force immediate termination |
| <code>kill -</code>  | Send a specific signal by number or name   |

#### Common Signals (Most used):

| Signal No. | Signal Name | What it does                                                                            |
|------------|-------------|-----------------------------------------------------------------------------------------|
| 1          | SIGHUP      | Reload configuration — tells a service to reread its config without restarting          |
| 2          | SIGINT      | Interrupt — same as pressing Ctrl+C in a terminal                                       |
| 15         | SIGTERM     | Graceful shutdown request — process can clean up and exit normally. Default signal.     |
| 9          | SIGKILL     | Forced immediate termination — cannot be ignored or blocked. Kernel kills it instantly. |
| 19         | SIGSTOP     | Pause/freeze a process — cannot be ignored                                              |
| 18         | SIGCONT     | Resume a paused process                                                                 |

#### SIGTERM vs SIGKILL — the important difference:

- **SIGTERM (15)** — politely asks the process to shut down. It can finish what it is doing, close files cleanly, and exit gracefully. Always try this first!
- **SIGKILL (9)** — the kernel forcibly kills the process immediately, no questions asked. The process has no chance to clean up. Use only when SIGTERM fails.

| Command                         | What it does                              |
|---------------------------------|-------------------------------------------|
| <code>kill 1042</code>          | Send SIGTERM — polite request to stop     |
| <code>kill -15 1042</code>      | Same as above — explicit SIGTERM          |
| <code>kill -9 1042</code>       | SIGKILL — forced termination, last resort |
| <code>kill -SIGKILL 1042</code> | Same as -9, using name instead of number  |

#### Why not always use -9?

Forcing a process to die instantly can leave things in a bad state — open files not closed properly, database transactions not committed, temporary files not cleaned up. SIGTERM gives the process a chance to tidy up first. Only reach for -9 when SIGTERM fails.

To see all available signals:

```
kill -l
```

## 5. pgrep and pkill

### pgrep - Finding Processes by Name

A cleaner alternative to `'ps aux | grep'` — returns just the PID of matching processes rather than the full process details.

**Why use it?** When you need a PID — for example to pass straight to a kill command — pgrep is faster and cleaner than reading the PID from ps aux output manually.

| Command                        | What it does                                                                                         |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <code>pgrep jellyfin</code>    | Returns just the PID of any process named jellyfin                                                   |
| <code>pgrep -l jellyfin</code> | Returns PID and process name                                                                         |
| <code>pgrep -a jellyfin</code> | Returns PID and full command line                                                                    |
| <code>pgrep -u scott</code>    | Returns all PIDs owned by user scott. N.B. Jellyfin PID does not show up — it runs in Docker as root |
| <code>pgrep -x jellyfin</code> | Exact match only — will only match 'jellyfin'                                                        |
| <code>ps -p 80192</code>       | Show details for a specific PID — e.g. PID 80192 is the Palworld game server                         |

#### Combining pgrep with kill:

```
kill -9 $(pgrep jellyfin) # Find PID and kill in one command.
# $() = command substitution - runs pgrep jellyfin first,
# takes the output (PID) and passes it directly to kill -9
```

#### pkill — Kill by Name Instead of PID

pkill combines the search and kill into one step — no need to find the PID first. When you know the process name, this saves two steps (`ps aux | grep`, then `kill`).

| Command                        | What it does                                 |
|--------------------------------|----------------------------------------------|
| <code>pkill jellyfin</code>    | Send SIGTERM to all processes named jellyfin |
| <code>pkill -9 jellyfin</code> | Send SIGKILL to all processes named jellyfin |

## 6. Real Time Process Monitoring

### top

A real time process viewer built into Linux. Unlike 'ps aux' which takes a snapshot, **top** updates continuously showing what is happening right now.

**Why use it?** When you need to see what is consuming CPU or Memory in real time — if the system feels slow or unresponsive, **top** immediately shows which process is the culprit.

To launch: `top`

#### What you see:

The top section shows a system summary:

- **Load average** — system load over the last 1, 5, and 15 minutes
- **Tasks** — total processes and how many are running, sleeping, stopped, zombie
- **%CPU** — how CPU time is being used
- **%MEM** — memory usage overview

The bottom section shows individual processes, sorted by CPU usage by default.

### Explanation of top headers:

| Header                 | Meaning                                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PID                    | Process ID.                                                                                                                                                                     |
| USER                   | Who owns the process.                                                                                                                                                           |
| PR (Priority)          | The kernel's internal priority value. Lower = higher priority = more CPU time. Calculated from NI value — you cannot set PR directly, only influence it via NI.                 |
| NI (Nice Value)        | User-adjustable priority hint. Range: -20 (highest priority) to 19 (lowest). Default is 0. High NI = process steps aside for others. Controlled by nice and renice. (See below) |
| VIRT (Virtual Memory)  | Total memory the process has reserved — RAM, swap, shared libraries, memory mapped files. Same as VSZ in ps aux. Always larger than RES.                                        |
| RES (Resident Memory)  | Actual physical RAM in use right now. Same as RSS in ps aux. This is the real number that matters.                                                                              |
| SHR (Shared Memory)    | Portion of RES shared with other processes — typically shared libraries like libc. Exclusive memory = RES minus SHR.                                                            |
| S (Process State)      | R=Running, S=Sleeping, D=Uninterruptable Sleep (hardware I/O), T=Stopped, Z=Zombie                                                                                              |
| %CPU                   | Current CPU usage. Can exceed 100% on multi-core systems — two full cores = 200%.                                                                                               |
| %MEM                   | RES as a % of total RAM.                                                                                                                                                        |
| TIME+ (Total CPU Time) | Cumulative CPU time since process started (minutes:seconds:hundredths). Unlike %CPU (current), TIME+ is a running total.                                                        |
| COMMAND                | Program that launched the process. Often truncated — press c inside top to toggle full command line.                                                                            |

### Useful keys while inside top:

| Key | What it does                                 |
|-----|----------------------------------------------|
| q   | Quit top                                     |
| k   | Kill a process — prompts for PID then signal |
| M   | Sort by memory usage                         |
| P   | Sort by CPU usage (default)                  |
| u   | Filter by username                           |
| h   | Help                                         |
| 1   | Show individual CPU cores                    |

## htop - The Friendlier Version of top

**htop** does the same job as **top** but with a colour-coded, more readable interface — CPU cores as progress bars, memory as a visual bar, mouse support, and easy scrolling. Not always installed by default.

```
sudo apt install htop # If not already installed
htop # Launch htop
```

| Key | What it does                                 |
|-----|----------------------------------------------|
| q   | Quit                                         |
| F9  | Kill a process                               |
| F6  | Sort by column                               |
| F4  | Filter processes by name                     |
| F5  | Tree view — shows parent/child relationships |
| u   | Filter by user                               |

## 7. Nice and Renice — Controlling Process Priority

Sometimes you need to run a resource intensive task — a large backup, a file conversion, a system scan — but you do not want all other processes slowing to a crawl. **nice** and **renice** let you control how much CPU priority a process gets, so other services keep running smoothly while a background task runs at lower priority.

### nice — Set Priority When Starting a Process

| Command                                  | What it does                                 |
|------------------------------------------|----------------------------------------------|
| nice -n 10 tar -czf backup.tar.gz ~/home | Run backup at low priority (NI 10)           |
| nice -n -10 ./important-script.sh        | Run at high priority (NI -10, requires sudo) |
| nice -n 19 apt upgrade                   | Run at lowest possible priority (NI 19)      |

| Option / Term | Meaning                                                                                     |
|---------------|---------------------------------------------------------------------------------------------|
| -n            | Set the nice value — e.g. nice -n 19                                                        |
| tar           | Command that can create and extract an archive                                              |
| .tar          | The file format/extension of the archive                                                    |
| .gz           | gzip compression applied to the archive (.tar) — so .tar.gz shows the archive is compressed |
| -c            | Create a new archive                                                                        |
| -z            | gzip — compress the archive using gzip                                                      |
| -f            | File — specifies the filename of the archive that follows                                   |
| .tgz          | You may see this — means exactly the same as .tar.gz, just shorter                          |

#### NI Value Reminder:

- **-20** = Highest priority, gets the most CPU
- **0** = Default
- **19** = Lowest priority, yields to everything else



■■ **Important:** By default, a regular user can only set a nice value between 0 and 19 — meaning you can only make a process less important than normal. To give a process higher priority (a negative value), you need sudo. Only root has this ability. Remember NI influences PR — the kernel's internal priority value. Formula: **PR = 20 + NI**. PR starts at 20 and NI adjusts it: NI 0 = PR stays at 20 (default) | NI 19 = PR goes to 39 (lower priority) | NI -20 = PR goes to 0 (highest priority). This explains why the NI range is -20 to 19.

## Renice — Change Priority of a Running Process

The nice command is only used when **starting** a process. Use **renice** to change the priority of a process that is already running.

| Command                   | What it does                                     |
|---------------------------|--------------------------------------------------|
| renice -n 10 -p 1974      | Change process 1974 to nice value 10             |
| renice -n 19 -p 1974      | Drop process 1974 to lowest priority             |
| sudo renice -n -5 -p 1974 | Increase priority (requires sudo)                |
| -n                        | The new nice value to set (used by nice as well) |
| -p                        | Target a specific process by PID                 |
| -u                        | Change priority of all processes owned by a user |

### Practical Example:

```
pgrep -a palworld # Find the Palworld process PID
renice -n 15 -p # Drop it to low priority
```

## 8. jobs, bg, fg and nohup — Managing Foreground and Background Processes

Sometimes when you start a command from the terminal it can take a long time — a file transfer, a large compression, a system scan. By default it runs in the **FOREGROUND**, blocking the terminal until it finishes. The following commands control whether a process runs in the FOREGROUND or BACKGROUND, and lets you switch between the two.

### Command and Process:

- **Command:** What you type into the terminal to launch something.
- **Process:** What gets created by the kernel when the command runs — the running instance of the program.

### Foreground VS Background:

- **Foreground** – the process has control of the terminal. You cannot type anything else until it finishes or you interrupt it (Ctrl+C).
- **Background** – the process runs silently behind the scenes. Your terminal is free for other things.

## & — Send a Process to the Background at Launch

Add & to the end of any command to start it in the background immediately:

| Example                         | What it does              |
|---------------------------------|---------------------------|
| tar -czf backup.tar.gz ~/home & | Run backup in background  |
| apt upgrade &                   | Run upgrade in background |

When you add '&' to a command, bash prints a job number and PID: [1] 1974 — [1] is the job number, 1974 is the PID.

## jobs — List Background Jobs

Shows all processes currently running in the background in your current shell session. Command to run: `jobs`

```
[1]+ Running tar -czf backup.tar.gz ~/home
[2]- Stopped apt upgrade
```

| Symbol          | Meaning                                                         |
|-----------------|-----------------------------------------------------------------|
| [1]             | Job Number                                                      |
| +               | Current Job — most recent (fg would bring this back by default) |
| -               | Previous Job                                                    |
| Running/Stopped | Current State                                                   |

There can only be one + and one - at any time. When you add a new process to the background it becomes the new +, and the previous + becomes -.

## fg — Bring a Background Job to the Foreground

This command brings a background job to the foreground — your terminal will be taken over by the process.

### Why would you want to do this?

- **To interact with it** — some processes need user input at some point
- **To monitor output live** — watch what it is printing in real time
- **To cancel it** — Ctrl+C only works on foreground processes. Bring it forward first, then Ctrl+C.
- **To pause it** — Ctrl+Z sends SIGSTOP, pausing it and returning you to the terminal. Process is frozen, not dead. Then: **fg** to resume in foreground, or **bg** to resume in background.
- **To check progress** — bring it forward, see what it is printing, then Ctrl+Z and bg to send it back.

### Example — several jobs in background:

| Job     | State                                           | Command                  |
|---------|-------------------------------------------------|--------------------------|
| [1]     | Running                                         | apt upgrade &            |
| [2]-    | Stopped                                         | tar -czf backup.tar.gz & |
| [3]+    | Running                                         | ./scan.sh &              |
| [4]     | Stopped                                         | ./script.sh &            |
| [5]     | Running                                         | ping google.com &        |
| Command | What it does                                    |                          |
| fg      | Bring the most recent (+) job to the foreground |                          |
| fg %1   | Bring job #1 (apt upgrade) to foreground        |                          |
| fg %5   | Bring job #5 (ping google.com) to foreground    |                          |

## nohup — Run a Process That Survives Logout

Nohup (no hangup) runs a command that keeps running even after you log out or close your SSH session. Without tmux or nohup, closing your SSH connection kills any running foreground processes.

When you log out, you break connection with that terminal session. The nohup command (running in the background) saves the output of the process to a file. Once you log back in, because you are in a different terminal session you cannot bring the process back to the foreground — but you can read what it has been doing. The output file is appended continuously while the process runs, surviving multiple logouts. Always use nohup with & to send it to the background.

| Command                 | What it does                                             |
|-------------------------|----------------------------------------------------------|
| nohup ./log-script.sh & | Run script — survives logout, output saved to nohup.out  |
| cat nohup.out           | Once logged back in, see what the process has been doing |

Output is saved to nohup.out in the current directory — you can check it later.

■■ To stop a nohup process after logging back in, find its PID with pgrep and use kill to terminate it. You cannot use fg — the terminal session link was broken when you logged out.

## ■ Linux+ Exam Tips Summary

- ps aux (BSD style, no dashes) and ps -ef (UNIX/POSIX style, with dashes) show similar information in different formats — know both
- ps aux shows %CPU, %MEM and STAT. ps -ef shows PPID. Neither shows everything — use the right one for the job
- grep headers disappear when piped — use grep -E "USER|pattern" to show the header alongside results
- awk processes by column (\$1, \$2, \$3...). Use ps aux | awk '\$8=="Z"' to find zombies in STAT column 8
- Zombie processes cannot be killed directly — kill the parent process instead
- SIGTERM (15) = graceful shutdown. SIGKILL (9) = forced kill. Always try SIGTERM first
- SIGHUP (1) = reload config without restarting — useful for applying config changes to running services
- kill sends signals — it does not only kill. kill -l lists all 64 signals
- pgrep finds by name and returns PID. pkill finds and kills by name in one step
- top is built-in and always available. htop needs installing but is easier to use
- NI range is -20 (highest priority) to 19 (lowest). Default is 0. Formula: PR = 20 + NI
- Only root can set negative NI values (higher priority than default)
- nice sets priority at launch. renice changes priority of an already running process
- & sends a process to the background at launch. jobs lists background jobs in current session
- Ctrl+Z pauses (SIGSTOP) a foreground process. fg resumes in foreground. bg resumes in background
- nohup keeps a process running after logout. Output goes to nohup.out. Cannot use fg after logout
- tmux is generally preferred over nohup — you can reattach and interact with the process